تشغيل التطبيق وربط المكونات

main.go

```go
func main() {
    database.ConnDB()
    database.Migrate()
    database.SeedAdmin()
    database.SeedCategories()
    database.SeedItems()

    app := fiber.New()
    app.Static("/upload", "./upload")

    routers.RegisterAuthRoutes(app)
    routers.RegisterChatRoutes(app)
    routers.RegisterCategoryRoutes(app)
    routers.RegisterFavoriteRoutes(app)
    routers.RegisterItemRoutes(app)
    routers.RegisterReviewRoutes(app)
    routers.RegisterUploadRoutes(app)
    routers.RegisterViewRoutes(app)

    port := os.Getenv("APP_PORT")
    if port == "" {
        port = "4000"
    }

    if err := app.Listen(":" + port); err != nil {
        panic(err)
    }
}
```

شرح الجزء:

هذه الدالة هي نقطة بداية مشروع TrendFlix. في البداية يتم الاتصال بقاعدة البيانات، ثم تنفيذ الـ migration وإنشاء بيانات أولية مثل المدير والتصنيفات والعناصر. بعد ذلك يتم إنشاء تطبيق Fiber، فتح مسار ملفات الرفع، وتسجيل كل مجموعات الـ routes الخاصة بالتسجيل، العناصر، المفضلة، التقييمات، الشات، والواجهات. في النهاية يقرأ التطبيق رقم المنفذ من APP_PORT، وإذا لم يكن موجودًا يعمل افتراضيًا على المنفذ 4000.

الاتصال بقاعدة البيانات

database/db_conn.go

```go
19  func ConnDB() {
20      if err := godotenv.Load(); err != nil {
21          log.Println(".env file not found, using environment variables")
22      }
23
24      dsn := strings.TrimSpace(os.Getenv("DB_DSN"))
25      if dsn == "" {
26          dbName := strings.TrimSpace(os.Getenv("DB_NAME"))
27          dbUser := strings.TrimSpace(os.Getenv("DB_USER"))
28          dbPassword := os.Getenv("DB_PASSWORD")
29          dbHost := strings.TrimSpace(os.Getenv("DB_HOST"))
30          dbPort := strings.TrimSpace(os.Getenv("DB_PORT"))
31
32          if dbName == "" || dbUser == "" || dbHost == "" || dbPort == "" {
33              panic("database configuration is incomplete")
34          }
35
36          dsn = fmt.Sprintf(
37              "%s:%s@tcp(%s:%s)/%s?charset=utf8mb4&parseTime=True&loc=Local",
38              dbUser,
39              dbPassword,
    ...
42              dbName,
43          )
44      }
45
46      db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{
47          Logger: logger.Default.LogMode(logger.Error),
48      })
49      if err != nil {
50          panic(fmt.Sprintf("database connection failed: %v", err))
51      }
52
```

شرح الجزء:

هذا الجزء مسؤول عن تجهيز اتصال MySQL باستخدام GORM. الكود يحاول قراءة ملف .env، ثم يبني DSN من متغيرات البيئة إذا لم يكن DB_DSN جاهزًا. بعد ذلك يفتح الاتصال، يتأكد من أن handle الخاص بقاعدة البيانات صالح، ويجرب Ping للتأكد أن السيرفر متاح. عند نجاح كل ذلك يتم تخزين الاتصال في DbConn ليستخدمه باقي المشروع.

نموذج العنصر داخل النظام

models/item_model.go

```go
type Item struct {
    ID          uint       `gorm:"column:id;primaryKey;autoIncrement" json:"id"`
    Title       string     `gorm:"column:title;not null" json:"title"`
    Description string     `gorm:"column:description;type:text" json:"description"`
    Type        string     `gorm:"column:type;not null" json:"type"`
    CoverImage  string     `gorm:"column:cover_image" json:"cover_image"`
    ContentLink *string    `gorm:"column:content_link" json:"content_link"`
    ReleaseDate time.Time  `gorm:"column:release_date;type:date" json:"release_date"`
    Author      *string    `gorm:"column:author" json:"author"`
    Director    *string    `gorm:"column:director" json:"director"`
    Developer   *string    `gorm:"column:developer" json:"developer"`
    Duration    *uint      `gorm:"column:duration" json:"duration"`
    PagesCount  *uint      `gorm:"column:pages_count" json:"pages_count"`
    Platform    *string    `gorm:"column:platform" json:"platform"`
    Rating      float64    `gorm:"column:rating" json:"rating"`
    Categories  []Category
        `gorm:"many2many:category_item;joinForeignKey:ItemID;joinReferences:CategoryID"
        json:"categories,omitempty"`
    CreatedAt   time.Time  `gorm:"column:created_at;autoCreateTime" json:"created_at"`
    UpdatedAt   time.Time  `gorm:"column:updated_at;autoUpdateTime" json:"updated_at"`
}

func (Item) TableName() string {
```

شرح الجزء:

Model Item يمثل أي محتوى داخل TrendFlix سواء كان فيلمًا أو لعبة أو كتابًا. يحتوي على العنوان، الوصف، النوع، صورة الغلاف، رابط المحتوى، تاريخ الإصدار، وحقول تختلف حسب النوع مثل المؤلف للكتب أو المخرج للأفلام أو المطور للألعاب. وجود Categories بعلاقة many-to-many يسمح للعنصر الواحد أن ينتمي لأكثر من تصنيف.

إنشاء مستخدم جديد

controller/auth/auth_controller.go

```go
    result := database.DbConn.Where("email = ?", request.Email).First(&existingUser)
    if result.Error != nil && !errors.Is(result.Error, gorm.ErrRecordNotFound) {
        log.Println("Error querying database:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }
    if result.RowsAffected > 0 {
        context["statusText"] = "bad"
        context["msg"] = "Email already exists"
        return c.Status(fiber.StatusConflict).JSON(context)
    }

    hashedPassword, err := bcrypt.GenerateFromPassword([]byte(request.Password), bcrypt.DefaultCost)
    if err != nil {
        log.Println("Error hashing password:", err)
        context["statusText"] = "bad"
        context["msg"] = "Error hashing password"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    record := models.User{
        Name:     request.Name,
    ...

    result = database.DbConn.Create(&record)
    if result.Error != nil {
        log.Println("Error in saving data:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Error in saving user"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    token, err := generateAuthToken(record)
```

شرح الجزء:

هذا الجزء من CreateUser يعالج عملية التسجيل. يتم تنظيف الاسم والبريد وكلمة المرور، ثم التحقق أن البيانات الأساسية موجودة. بعد ذلك يبحث في قاعدة البيانات لمنع تكرار البريد. إذا كان البريد جديدًا، يتم تشفير كلمة المرور باستخدام bcrypt قبل الحفظ، ثم إنشاء سجل المستخدم بدور user. هذه الخطوات تحمي كلمة المرور وتمنع إنشاء حسابات مكررة بنفس البريد.

تسجيل الدخول وإنشاء JWT

controller/auth/auth_controller.go

```go
    }

    var user models.User
    result := database.DbConn.Where("email = ?", request.Email).First(&user)
    if result.Error != nil {
        if errors.Is(result.Error, gorm.ErrRecordNotFound) {
            context["statusText"] = "bad"
            context["msg"] = "Invalid email or password"
            return c.Status(fiber.StatusUnauthorized).JSON(context)
        }
        log.Println("Error querying database:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    ...
func generateAuthToken(user models.User) (string, error) {
    secret := strings.TrimSpace(os.Getenv("JWT_SECRET"))
    if secret == "" {
        return "", errors.New("JWT_SECRET is not configured")
    }

    now := time.Now()
    claims := jwt.MapClaims{
        "sub":  strconv.FormatUint(uint64(user.ID), 10),
        "role": user.Role,
        "iat":  now.Unix(),
        "exp":  now.Add(24 * time.Hour).Unix(),
    }

```

شرح الجزء:

LoginUser يستقبل البريد وكلمة المرور، ينظف المدخلات، ثم يبحث عن المستخدم في قاعدة البيانات. إذا لم يوجد المستخدم أو كانت كلمة المرور غير صحيحة يرجع Unauthorized. عند نجاح المطابقة يتم إنشاء JWT يحتوي على رقم المستخدم والدور ووقت الانتهاء. هذا التوكن هو ما يستخدمه الفرونت إند لاحقًا للوصول إلى الصفحات والعمليات المحمية.

التحقق من التوكن وحماية الطلبات

middleware/auth_middleware.go

```go
30
31      authorizationHeader := strings.TrimSpace(c.Get("Authorization"))
32      if authorizationHeader == "" || !strings.HasPrefix(authorizationHeader, "Bearer ") {
33          return c.Status(fiber.StatusUnauthorized).JSON(context)
34      }
35
36      tokenString := strings.TrimSpace(strings.TrimPrefix(authorizationHeader, "Bearer "))
37      if tokenString == "" {
38          return c.Status(fiber.StatusUnauthorized).JSON(context)
39      }
40
41      secret := strings.TrimSpace(os.Getenv("JWT_SECRET"))
42      if secret == "" {
43          log.Println("JWT_SECRET is not configured")
        ...
45          return c.Status(fiber.StatusInternalServerError).JSON(context)
46      }
47
48      claims := jwt.MapClaims{}
49      token, err := jwt.ParseWithClaims(tokenString, claims, func(token *jwt.Token) (interface{}, error) {
50          if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
51              return nil, fmt.Errorf("unexpected signing method: %v", token.Header["alg"])
52          }
        ...
74              return c.Status(fiber.StatusUnauthorized).JSON(context)
75          }
76
77          log.Println("Error querying authenticated user:", result.Error)
78          context["msg"] = "Database error"
79          return c.Status(fiber.StatusInternalServerError).JSON(context)
80      }
81
82      c.Locals("currentUser", user)
83      return c.Next()
```

شرح الجزء:

Middleware Authenticate يحمي الـ APIs التي تحتاج مستخدمًا مسجلًا. يقرأ Authorization header ويتأكد أن الصيغة Bearer token صحيحة، ثم يستخدم JWT_SECRET للتحقق من صحة التوقيع. بعد ذلك يستخرج user id من claim sub ويبحث عن المستخدم في قاعدة البيانات. عند النجاح يخزن المستخدم في Locals باسم currentUser حتى تستخدمه الـ controllers التالية.

صلاحيات المدير ومسارات العناصر

routers/item_router.go

```go
func RegisterItemRoutes(app *fiber.App) {
    items := app.Group("/items")
    items.Get("", itemcontroller.GetItems)
    items.Get("/:id", itemcontroller.GetItemByID)

    adminItems := app.Group("/items", middleware.Authenticate, middleware.RequireAdmin)
    adminItems.Post("", itemcontroller.CreateItem)
    adminItems.Put("/:id", itemcontroller.UpdateItem)
```

شرح الجزء:

هذا الـ router يقسم عمليات العناصر إلى قسمين. عرض كل العناصر أو عنصر واحد متاح بدون حماية، لأن المستخدم يحتاج تصفح الكتالوج. أما إنشاء عنصر أو تعديله أو حذفه فيمر عبر Authenticate ثم RequireAdmin، لذلك لا يستطيع تنفيذ هذه العمليات إلا مستخدم مسجل ودوره admin.

جلب قائمة العناصر من قاعدة البيانات

controller/item_controller/item_controller.go

```go
func GetItems(c *fiber.Ctx) error {
    context := fiber.Map{
        "statusText": "Ok",
        "msg":        "Items fetched successfully",
    }

    if database.DbConn == nil {
        log.Println("database connection is not initialized")
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    var items []models.Item
    result := database.DbConn.Preload("Categories").Order("id DESC").Find(&items)
    if result.Error != nil {
        log.Println("Error fetching items:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    context["items"] = items
    return c.Status(fiber.StatusOK).JSON(context)
```

شرح الجزء:

GetItems يرجع قائمة العناصر كاملة مرتبة من الأحدث إلى الأقدم، ويستخدم Preload لتحميل التصنيفات المرتبطة بكل عنصر. قبل الاستعلام يتحقق أن اتصال قاعدة البيانات موجود، وإذا حدث خطأ في الاستعلام يرجع استجابة JSON بحالة فشل ورسالة Database error. عند النجاح يضع العناصر داخل context باسم items ويرجعها للواجهة.

إنشاء عنصر باستخدام Transaction

controller/item_controller/item_controller.go

```go
        context["statusText"] = "bad"
        context["msg"] = "Invalid request"
        return c.Status(fiber.StatusBadRequest).JSON(context)
    }

    record, msg, statusCode := buildItemFromRequest(request)
    if statusCode != 0 {
        context["statusText"] = "bad"
        context["msg"] = msg
        return c.Status(statusCode).JSON(context)
    }

    categories, msg, statusCode := loadCategoriesByIDs(request.CategoryIDs)
    if statusCode != 0 {
    ...
        return c.Status(statusCode).JSON(context)
    }

    tx := database.DbConn.Begin()
    if tx.Error != nil {
        log.Println("Error starting transaction:", tx.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    if err := tx.Create(&record).Error; err != nil {
        tx.Rollback()
        log.Println("Error saving item:", err)
        context["statusText"] = "bad"
```

شرح الجزء:

CreateItem مسؤول عن إضافة فيلم أو لعبة أو كتاب جديد. بعد التأكد من وجود اتصال وقبول المدير، يقرأ جسم الطلب ثم يبني العنصر ويتحقق من التصنيفات. استخدام transaction مهم هنا لأن إنشاء العنصر وربطه بالتصنيفات يجب أن ينجحا معًا. إذا فشلت أي خطوة يتم Rollback، وإذا نجحت كل الخطوات يتم Commit وإرجاع العنصر الجديد.

تنظيف بيانات العنصر حسب النوع

controller/item_controller/item_controller.go

```go
func buildItemFromRequest(request itemRequest) (models.Item, string, int) {
    request.Title = strings.TrimSpace(request.Title)
    request.Description = strings.TrimSpace(request.Description)
    request.Type = strings.ToLower(strings.TrimSpace(request.Type))
    request.CoverImage = strings.TrimSpace(request.CoverImage)
    request.ContentLink = normalizeOptionalString(request.ContentLink)
    request.ReleaseDate = strings.TrimSpace(request.ReleaseDate)
    request.Author = normalizeOptionalString(request.Author)
    request.Director = normalizeOptionalString(request.Director)
    request.Developer = normalizeOptionalString(request.Developer)
    request.Platform = normalizeOptionalString(request.Platform)

    if request.Title == "" || request.Type == "" || request.ReleaseDate == "" {
        return models.Item{}, "Title, type and release_date are required", fiber.StatusBadRequest
    }

    if request.Type != "book" && request.Type != "movie" && request.Type != "game" {
        return models.Item{}, "Type must be book, movie or game", fiber.StatusBadRequest
    }

    releaseDate, err := time.Parse("2006-01-02", request.ReleaseDate)
    ...
        Rating:      request.Rating,
    }

    switch item.Type {
    case "book":
        item.Director = nil
        item.Developer = nil
        item.Duration = nil
        item.Platform = nil
    case "movie":
        item.Author = nil
```

شرح الجزء:

buildItemFromRequest تحول بيانات الطلب إلى Model صالح للحفظ. الكود ينظف النصوص، يتحقق من الحقول المطلوبة، يقبل فقط الأنواع book أو movie أو game، ويتأكد من صيغة التاريخ. بعد إنشاء العنصر يستخدم switch لإزالة الحقول غير المناسبة لكل نوع؛ مثل حذف director من الكتاب أو author من الفيلم. هذا يحافظ على بيانات منظمة داخل قاعدة البيانات.

إنشاء تصنيف جديد

controller/categories_controller/categories_controller.go

```go
        log.Println("database connection is not initialized")
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    var request categoryRequest
    if err := c.BodyParser(&request); err != nil {
        log.Printf("Error parsing request body: %v", err)
        context["statusText"] = "bad"
        context["msg"] = "Invalid request"
        return c.Status(fiber.StatusBadRequest).JSON(context)
    }

    request.Name = strings.TrimSpace(request.Name)
    request.Slug = strings.TrimSpace(request.Slug)

    if request.Name == "" || request.Slug == "" {
        context["statusText"] = "bad"
        context["msg"] = "Name and slug are required"
        return c.Status(fiber.StatusBadRequest).JSON(context)
    }
    ...
    result := database.DbConn.Where("slug = ?", request.Slug).First(&existingCategory)
    if result.Error != nil && !errors.Is(result.Error, gorm.ErrRecordNotFound) {
        log.Println("Error querying database:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }
    if result.RowsAffected > 0 {
        context["statusText"] = "bad"
        context["msg"] = "Slug already exists"
        return c.Status(fiber.StatusConflict).JSON(context)
```

شرح الجزء:

CreateCategory يستقبل name و slug من الطلب، ثم ينظف القيم ويتأكد أن الحقلين موجودان. بعد ذلك يبحث عن slug مكرر لأن الرابط المختصر للتصنيف يجب أن يكون فريدًا. إذا وجد تكرارًا يرجع Conflict، أما إذا لم يوجد فيكمل حفظ التصنيف الجديد في قاعدة البيانات وإرجاع بياناته.

قائمة المفضلة للمستخدم

controller/favorites_controller/favorites_controller.go

```go
func GetFavorites(c *fiber.Ctx) error {
    context := fiber.Map{
        "statusText": "Ok",
        "msg":        "Favorites fetched successfully",
    }

    if database.DbConn == nil {
        log.Println("database connection is not initialized")
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    user, err := currentUserFromContext(c, context)
    if err != nil {
        return err
    }

    var items []models.Item
    result := database.DbConn.
        Model(&models.Item{}).
        Joins("JOIN favorites ON favorites.item_id = items.id").
        Where("favorites.user_id = ?", user.ID).
        Preload("Categories").
        Order("favorites.created_at DESC").
        Find(&items)
    if result.Error != nil {
        log.Println("Error fetching favorites:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    context["items"] = items
```

شرح الجزء:

GetFavorites تعرض العناصر التي أضافها المستخدم الحالي إلى المفضلة. الدالة تحصل على المستخدم من currentUser الذي وضعه الـ middleware، ثم تنفذ JOIN بين جدول items وجدول favorites بناءً على user_id. كذلك يتم تحميل التصنيفات وترتيب النتائج حسب تاريخ الإضافة للمفضلة من الأحدث إلى الأقدم.

إضافة عنصر إلى المفضلة

controller/favorites_controller/favorites_controller.go

```
        context["msg"] = "Item already in favorites"
        return c.Status(fiber.StatusConflict).JSON(context)
    }
    if result.Error != nil && !errors.Is(result.Error, gorm.ErrRecordNotFound) {
        log.Println("Error querying favorite:", result.Error)
        context["statusText"] = "bad"
        context["msg"] = "Database error"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    favorite = models.Favorite{
        UserID: user.ID,
        ItemID: itemID,
    }

    if err := database.DbConn.Create(&favorite).Error; err != nil {
        log.Println("Error saving favorite:", err)
        context["statusText"] = "bad"
        context["msg"] = "Error saving favorite"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    context["id"] = favorite.ID
```

شرح الجزء:

AddFavorite تضيف عنصرًا للمفضلة بطريقة آمنة. أولًا تتأكد من المستخدم الحالي ومن رقم العنصر، ثم تتحقق أن العنصر موجود فعلًا. بعد ذلك تبحث هل نفس المستخدم أضاف نفس العنصر سابقًا؛ إذا كان موجودًا ترجع Conflict بدل تكرار السجل. إذا لم يكن موجودًا تنشئ سجل Favorite يحتوي على user id و item id.

إنشاء تقييم للمحتوى

controller/reviews_controller/reviews_controller.go

```go
        return err
    }

    var request createReviewRequest
    if err := c.BodyParser(&request); err != nil {
        log.Printf("Error parsing request body: %v", err)
        context["statusText"] = "bad"
        context["msg"] = "Invalid request"
        return c.Status(fiber.StatusBadRequest).JSON(context)
    }

    review, msg, statusCode := buildReviewFromCreateRequest(request)
    if statusCode != 0 {
        context["statusText"] = "bad"
        context["msg"] = msg
        return c.Status(statusCode).JSON(context)
    }

    if err := ensureItemExists(review.ItemID, context, c); err != nil {
        return err
    }

    var existingReview models.Review
    result := database.DbConn.Where("user_id = ? AND item_id = ?", user.ID,
        review.ItemID).First(&existingReview)
    if result.Error == nil {
        context["statusText"] = "bad"
        context["msg"] = "You already reviewed this item"
        return c.Status(fiber.StatusConflict).JSON(context)
```

شرح الجزء:

CreateReview يسمح للمستخدم بتقييم عنصر معين. الكود يقرأ الطلب، يتحقق أن التقييم بين 1 و5 وأن item_id موجود، ثم يتأكد أن العنصر موجود في قاعدة البيانات. بعد ذلك يمنع المستخدم من تقييم نفس العنصر أكثر من مرة. عند نجاح الشروط يربط التقييم بالمستخدم الحالي ويحفظه.

خدمة الشات داخل TrendFlix

controller/chat_controller/chat_controller.go

```go
        log.Println("OPENROUTER_API_KEY is not configured")
        contextMap["statusText"] = "bad"
        contextMap["msg"] = "Chat service is unavailable right now"
        return c.Status(fiber.StatusServiceUnavailable).JSON(contextMap)
    }

    var request chatRequest
    if err := c.BodyParser(&request); err != nil {
        contextMap["statusText"] = "bad"
        contextMap["msg"] = "Invalid request"
        return c.Status(fiber.StatusBadRequest).JSON(contextMap)
    }

    message := strings.TrimSpace(request.Message)
    if message == "" {
        contextMap["statusText"] = "bad"
        contextMap["msg"] = "Message is required"
        return c.Status(fiber.StatusBadRequest).JSON(contextMap)
    }

    if len([]rune(message)) > maxMessageLength {
        contextMap["statusText"] = "bad"
        contextMap["msg"] = "Message is too long"
        return c.Status(fiber.StatusBadRequest).JSON(contextMap)
    }

    messages := []openRouterEntry{
        {Role: "system", Content: trendFlixSystemPrompt},
```

شرح الجزء:

Reply هي نقطة API الخاصة بالمساعد الذكي داخل التطبيق. تتحقق أولًا من وجود OPENROUTER_API_KEY، ثم تقرأ رسالة المستخدم وتمنع الرسائل الفارغة أو الطويلة جدًا. بعد ذلك تبني رسائل النظام التي تحدد أن المساعد يجيب فقط عن الأفلام والألعاب والكتب، وتضيف سجل المحادثة والرسالة الجديدة قبل تجهيز طلب OpenRouter.

تنظيم سجل الشات واللغة

controller/chat_controller/chat_controller.go

```go
174 func normalizeHistory(history []openRouterEntry) []openRouterEntry {
175     if len(history) == 0 {
176         return nil
177     }
178
179     start := 0
180     if len(history) > maxChatHistoryMessages {
181         start = len(history) - maxChatHistoryMessages
182     }
183
184     normalized := make([]openRouterEntry, 0, len(history)-start)
185     for _, entry := range history[start:] {
186         role := strings.TrimSpace(strings.ToLower(entry.Role))
187         if role != "user" && role != "assistant" {
188             continue
189         }
190
191         content := strings.TrimSpace(entry.Content)
    ...
216 func buildLanguageInstruction(message string) string {
217     if isArabicText(message) {
218         return "Respond in Arabic because the user's latest message is in Arabic. If the request is
                outside movies, games, or books, politely refuse in Arabic and redirect to those topics."
219     }
220
221     return "Respond in English because the user's latest message is in English or another non-Arabic
            language. If the request is outside movies, games, or books, politely refuse in English and
            redirect to those topics."
222 }
223
224 func isArabicText(value string) bool {
225     for _, r := range value {
```

شرح الجزء:

normalizeHistory يقلل سجل المحادثة إلى آخر عدد محدد من الرسائل، ويتجاهل أي role غير user أو assistant، ويقص النص الطويل. هذا يمنع إرسال بيانات كثيرة إلى خدمة الذكاء الاصطناعي. أما buildLanguageInstruction و isArabicText فيحددان لغة الرد حسب آخر رسالة؛ فإذا كانت عربية يطلب من المساعد الرد بالعربية، وإلا يرد بالإنجليزية.

رفع الصور والتحقق من نوع الملف

controller/upload_controller/upload_controller.go

```go
func uploadImage(c *fiber.Ctx, subDir string, successMessage string) error {
    context := fiber.Map{
        "statusText": "Ok",
        "msg":        successMessage,
    }

    fileHeader, err := c.FormFile("file")
    if err != nil {
        context["statusText"] = "bad"
        context["msg"] = "Image file is required"
        return c.Status(fiber.StatusBadRequest).JSON(context)
    }

    if err := validateImageFile(fileHeader); err != nil {
        context["statusText"] = "bad"
        context["msg"] = err.Error()
        return c.Status(fiber.StatusBadRequest).JSON(context)
    }

    storageDir := filepath.Join(uploadRootDir, subDir)
    if err := os.MkdirAll(storageDir, 0o755); err != nil {
        context["statusText"] = "bad"
        context["msg"] = "Error preparing upload directory"
        return c.Status(fiber.StatusInternalServerError).JSON(context)
    }

    fileName := buildUploadFileName(fileHeader.Filename)
    storagePath := filepath.Join(storageDir, fileName)
    if err := c.SaveFile(fileHeader, storagePath); err != nil {
        context["statusText"] = "bad"
```

شرح الجزء:

uploadImage يستخدم لرفع صورة المستخدم أو صورة العنصر. يستقبل الملف من form field باسم file، ثم يتحقق من الامتداد ونوع المحتوى حتى يقبل الصور فقط. بعد ذلك ينشئ مجلد التخزين إذا لم يكن موجودًا، يبني اسم ملف جديد باستخدام الوقت الحالي لتجنب التصادم، يحفظ الملف، ثم يرجع المسار العام الذي يستخدمه الفرونت إند لعرض الصورة.

عرض صفحات الواجهة والملفات الثابتة

routers/view_router.go

```go
func RegisterViewRoutes(app *fiber.App) {
    cwd, err := os.Getwd()
    if err != nil {
        cwd = "."
    }
    viewDir := filepath.Join(cwd, "view")

    // Frontend entry + static assets
    app.Get("/", func(c *fiber.Ctx) error {
        return c.SendFile(filepath.Join(viewDir, "index.html"))
    })

    // Backward-compatible route (older JS used /pages/auth.html)
    // Must be registered before the /pages static handler.
    app.Get("/pages/auth.html", func(c *fiber.Ctx) error {
        return c.Redirect("/pages/auth/auth.html", fiber.StatusTemporaryRedirect)
    })

    app.Get("/detail/:id", func(c *fiber.Ctx) error {
        return c.SendFile(filepath.Join(viewDir, "pages", "detail.html"))
    })

```

شرح الجزء:

RegisterViewRoutes يربط صفحات HTML والملفات الثابتة بتطبيق Fiber. الصفحة الرئيسية ترسل index.html، ويوجد redirect لمسار قديم خاص بصفحة تسجيل الدخول للحفاظ على التوافق. مسار detail/:id يعرض صفحة التفاصيل، بينما Static يفتح مجلد assets لملفات CSS و JavaScript ومجلد pages لباقي صفحات الواجهة.

فلترة الكتالوج وبناء كروت العرض

view/assets/js/app.js

```
function getFilteredItems(type) {
  const activeCategory = activeCategoryByType[type] || "all";
  const query = searchQuery.trim().toLowerCase();

  return getItemsByType(type).filter((item) => {
    const matchesCategory =
      activeCategory === "all" || (item.categories || []).some((category) => String(category.id) ===
          activeCategory);
    const matchesSearch = !query || String(item.title || "").toLowerCase().includes(query);
    return matchesCategory && matchesSearch;
  });
}
    ...
function card(item) {
  const favoriteLabel = t("app.toggleFavorite");
  const safeName = escapeHtml(item.title || "");
  const safeImg = escapeHtml(item.cover_image || getFallbackImage(item.title));
  const rating = item.rating ? `* ${item.rating}` : "";
  const releaseDate = formatDate(item.release_date);
  const isFavorite = favoriteItemIds.has(String(item.id));
  const categoryNames = (item.categories || [])
    .slice(0, 2)
    .map((c) => escapeHtml(c.name || ""))
    .filter(Boolean)
    .join(" • ");

  return `
    <article
      class="card-item"
      data-detail-url="${escapeHtml(getDetailHref(item.id))}"
      tabindex="0"
      role="link"
      aria-label="Open details for ${safeName}"
```

شرح الجزء:

في الواجهة الأمامية، getFilteredItems تطبق فلترة النوع والتصنيف والبحث النصي على العناصر. دالة card تبني HTML لكارت واحد يحتوي على الصورة والعنوان والتقييم والتاريخ والتصنيفات وزر المفضلة. هذه الدوال هي الأساس الذي يجعل الكتالوج قابلًا للبحث والتصفية، ويحول بيانات الـ API إلى عناصر مرئية يستطيع المستخدم فتح تفاصيلها أو إضافتها للمفضلة.

بناء صفحة تفاصيل العنصر

view/assets/js/detail.js

```
68   function getTypeMeta(type) {
69     const map = {
70       movie: { icon: "🎬", labelKey: "detail.typeMovie", actionKey: "common.watch", actionIcon: "▶" },
71       game:  { icon: "🎮", labelKey: "detail.typeGame",  actionKey: "common.play",  actionIcon: "🎮" },
72       book:  { icon: "📖", labelKey: "detail.typeBook",  actionKey: "common.read",  actionIcon: "📖" },
73     };
74     return map[type] || { icon: "🎬", labelKey: "detail.typeMovie", actionKey: "common.watch", actionIcon:
         "▶" };
75   }
76
       ...
117  function buildPage(item, reviews) {
118    const meta = getTypeMeta(item.type);
119    const safeImg = item.cover_image || getFallbackImage(item.title);
120    const safeTitle = escapeHtml(item.title || "");
121    const contentLink = String(item.content_link || "").trim();
122    const metaHtml = buildMeta(item);
123    const cats = buildCategories(item.categories);
124    const reviewsHtml = buildReviews(reviews);
125    const ratingVal = item.rating ? item.rating.toFixed(1) : "";
126
127    return `
128      <div class="detail-backdrop" style="background-image:url('${escapeHtml(safeImg)}')"></div>
129      <div class="detail-page">
130
131        <nav class="detail-topbar">
132          <button class="detail-back" id="backBtn" type="button">
133            ← <span>${escapeHtml(t("detail.back"))}</span>
134          </button>
135          <span class="detail-brand">TrendFlix</span>
136          <div class="lang-menu">
```

شرح الجزء:

detail.js يحول بيانات العنصر إلى صفحة تفاصيل كاملة. getTypeMeta تحدد النص والأيقونة وزر الإجراء حسب النوع: مشاهدة، قراءة، أو لعب. buildMeta و buildCategories و buildReviews تنسق بيانات إضافية مثل المخرج أو المؤلف والتصنيفات والتقييمات. buildPage يجمع كل هذه الأجزاء في HTML واحد يعرض الخلفية، الصورة، العنوان، التقييم، الأزرار، والوصف.